ASSIGNMENT – II

22UIT501 – WEB PROGRAMMING

CODE:

INDEX.HTML

```
<!doctype html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<link rel="icon" type="image/svg+xml" href="/favicon.svg" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>frontend</title>
</head>
<body>
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

SERVER.JS:

```
const express = require("express");
const destinationRoutes = require("./routes/destinationRoutes");
const tripRoutes = require("./routes/tripRoutes");
console.log("TRIP ROUTE LOADED ✅");
const hotelRoutes = require("./routes/hotelRoutes");
const touristPlaceRoutes = require("./routes/touristPlaceRoutes");
const bookingRoutes = require("./routes/bookingRoutes");
const cors = require("cors");
const mysql = require("mysql2/promise");
require("dotenv").config({ path: __dirname + "/.env" });
```

```
const authRoutes = require("./routes/authRoutes");
const app = express();
const guideRoutes = require("./routes/guideRoutes");
const notificationRoutes = require("./routes/notificationRoutes");
const adminRoutes = require("./routes/adminRoutes");
const adminCustomerRoutes = require("./routes/adminCustomerRoutes");
const adminGuideRoutes = require("./routes/adminGuideRoutes");
const adminDestinationRoutes = require("./routes/adminDestinationRoutes");
const adminHotelRoutes = require("./routes/adminHotelRoutes");
const adminTouristPlaceRoutes =
require("./routes/adminTouristPlaceRoutes");
const adminBookingRoutes = require("./routes/adminBookingRoutes");
const adminReviewRoutes = require("./routes/adminReviewRoutes");
const reviewRoutes = require("./routes/reviewRoutes");
const authMiddleware = require("./middleware/authMiddleware");
const { requireRole } = authMiddleware;
app.use(cors());
app.use(express.json());
app.use("/api/auth", authRoutes);
app.use("/api/destinations", destinationRoutes);
app.use("/api/trips", tripRoutes);
app.use("/api/guides", guideRoutes);
app.use("/api/hotels", hotelRoutes);
app.use("/api/bookings", bookingRoutes);
app.use("/api/notifications", notificationRoutes);
console.log("NOTIFICATION ROUTE LOADED ✅");
app.use("/api/tourist-places", touristPlaceRoutes);
app.get("/api/trips/test", (req, res) => {
res.json({ message: "Trip route is working ✅" });
```

```
});
app.use("/api/admin", authMiddleware, requireRole("admin"));
app.use("/api/admin", adminRoutes);
app.use("/api/admin/customers", adminCustomerRoutes);
app.use("/api/admin/guides", adminGuideRoutes);
app.use("/api/admin/destinations", adminDestinationRoutes);
app.use("/api/admin/hotels", adminHotelRoutes);
app.use("/api/admin/tourist-places", adminTouristPlaceRoutes);
app.use("/api/admin/bookings", adminBookingRoutes);
app.use("/api/admin/reviews", adminReviewRoutes);
app.use("/api/reviews", reviewRoutes);
console.log("DB HOST:", process.env.DB_HOST);
console.log("DB USER:", process.env.DB_USER);
console.log("DB NAME:", process.env.DB_NAME);
console.log("DB PASSWORD LOADED:", !!process.env.DB_PASSWORD);
const db = mysql.createPool({
host: process.env.DB_HOST,
user: process.env.DB_USER,
password: process.env.DB_PASSWORD,
database: process.env.DB_NAME
});
app.locals.db = db;
app.get("/", (req, res) => {
res.send("TRAVELMATE-NEW-SERVER-12345");
});
app.get("/api/test-db", async (req, res) => {
try {
const [rows] = await db.query("SELECT 1 AS result");
res.json({
```

```
message: "MySQL connected successfully ✅",
data: rows
});
} catch (error) {
console.error("Database error:", error.message);

res.status(500).json({
message: "MySQL connection failed ❌",
error: error.message
});
}
};
const PORT = process.env.PORT || 5000;
app.listen(PORT, () =>
console.log(`TravelMate backend running on http://localhost:${PORT}`);
});
```

APP.JSX:

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import { AuthProvider } from "./context/AuthContext.jsx";
import Navbar from "./components/Navbar";
import ProtectedRoute from "./components/ProtectedRoute";
import Home from "./pages/Home";
import Login from "./pages/Login";
import Register from "./pages/Register";
import Profile from "./pages/Profile";
import Destinations from "./pages/Destinations";
import TripPlanner from "./pages/TripPlanner";
import Hotels from "./pages/Hotels";
import TouristPlaces from "./pages/TouristPlaces";
```

```
import Guides from "./pages/Guides";
import CustomerBookings from "./pages/CustomerBookings";
import GuideDashboard from "./pages/GuideDashboard";
import GuideNotifications from "./pages/GuideNotifications";
import GuideRegister from "./pages/GuideRegister";
import AdminDashboard from "./pages/AdminDashboard";
import AdminCustomers from "./pages/AdminCustomers";
import AdminGuides from "./pages/AdminGuides";
import AdminDestinations from "./pages/AdminDestinations";
import AdminHotels from "./pages/AdminHotels";
import AdminTouristPlaces from "./pages/AdminTouristPlaces";
import AdminBookings from "./pages/AdminBookings";
import AdminReviews from "./pages/AdminReviews";
function App() {
return (
<BrowserRouter>
<AuthProvider>
<Navbar />
<Routes>
<Route path="/" element={<Home />} />
<Route path="/login" element={<Login />} />
<Route path="/register" element={<Register />} />
<Route path="/profile" element={<ProtectedRoute><Profile
/></ProtectedRoute>} />
<Route path="/destinations" element={<Destinations />} />
<Route path="/trip-planner" element={<ProtectedRoute
roles={["customer"]}><TripPlanner /></ProtectedRoute>} />
<Route path="/hotels/:destinationId?" element={<Hotels />} />
<Route path="/tourist-places/:destinationId?"
element={<TouristPlaces />} />
```

```
<Route path="/guides/:destinationId?" element={<Guides />} />
<Route path="/my-bookings" element={<ProtectedRoute
roles={["customer"]}><CustomerBookings /></ProtectedRoute>} />
<Route path="/customer-bookings" element={<ProtectedRoute
roles={["customer"]}><CustomerBookings /></ProtectedRoute>} />
<Route path="/guide-dashboard" element={<ProtectedRoute
roles={["guide"]}><GuideDashboard /></ProtectedRoute>} />
<Route path="/guide-notifications" element={<ProtectedRoute
roles={["guide"]}><GuideNotifications /></ProtectedRoute>} />
<Route path="/guide-register" element={<GuideRegister />} />
<Route path="/admin-dashboard" element={<ProtectedRoute
roles={["admin"]}><AdminDashboard /></ProtectedRoute>} />
<Route
path="/admin-customers"
element={<ProtectedRoute roles={["admin"]}><AdminCustomers
/></ProtectedRoute>}
/>
<Route path="/admin-guides" element={<ProtectedRoute
roles={["admin"]}><AdminGuides /></ProtectedRoute>} />
<Route path="/admin-destinations" element={<ProtectedRoute
roles={["admin"]}><AdminDestinations /></ProtectedRoute>} />
<Route path="/admin-hotels" element={<ProtectedRoute
roles={["admin"]}><AdminHotels /></ProtectedRoute>} />
<Route
path="/admin-tourist-places"
element={<ProtectedRoute roles={["admin"]}><AdminTouristPlaces
/></ProtectedRoute>}
/>
<Route
path="/admin-bookings"
element={<ProtectedRoute roles={["admin"]}><AdminBookings
/></ProtectedRoute>}
/>
```

```
<Route
path="/admin-reviews"
element={<ProtectedRoute roles={["admin"]}><AdminReviews
/></ProtectedRoute>}
/>
</Routes>
</AuthProvider>
</BrowserRouter>
);
}
export default App;
```

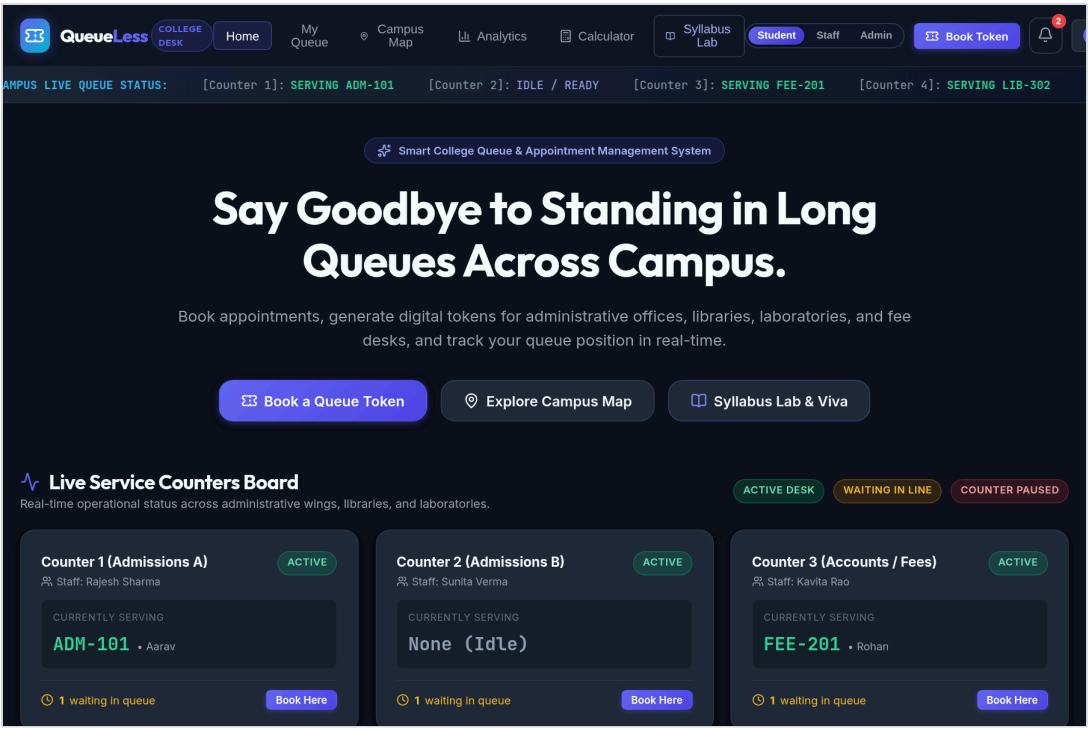
AUTHMIDDLEWARE.JS

```
const jwt = require("jsonwebtoken");
function authMiddleware(req, res, next) {
const authorization = req.headers.authorization;
if (!authorization || !authorization.startsWith("Bearer ")) {
return res.status(401).json({
message: "Authorization token required"
});
}
const token = authorization.slice(7).trim();
if (!token) {
return res.status(401).json({
message: "Authorization token required"
});
}
try {
req.user = jwt.verify(token, process.env.JWT_SECRET, {
```

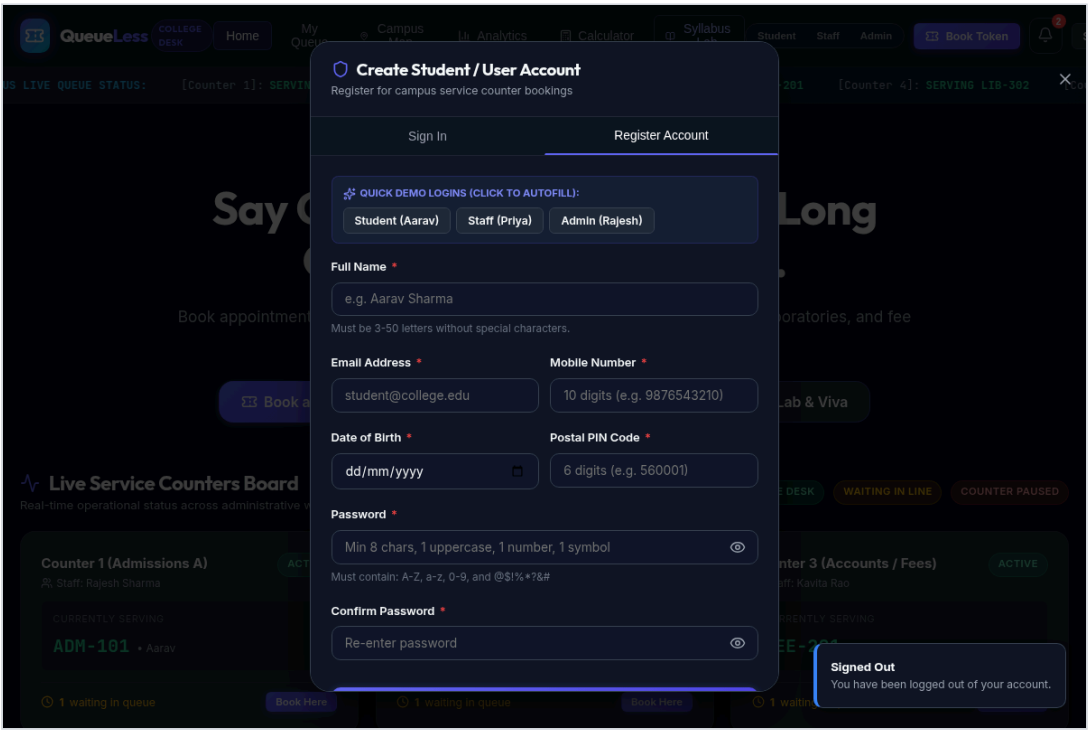
```
algorithms: ["HS256"]
});
return next();
} catch (error) {
return res.status(401).json({
message: "Invalid or expired token"
});
}
}
function requireRole(...allowedRoles) {
return (req, res, next) => {
if (!req.user) {
return res.status(401).json({
message: "Authentication required"
});
}
if (!allowedRoles.includes(req.user.role)) {
return res.status(403).json({
message: "You do not have permission to access this resource"
});
}
return next();
};
}
module.exports = authMiddleware;
module.exports.requireRole = requireRole;
```

OUTPUT:

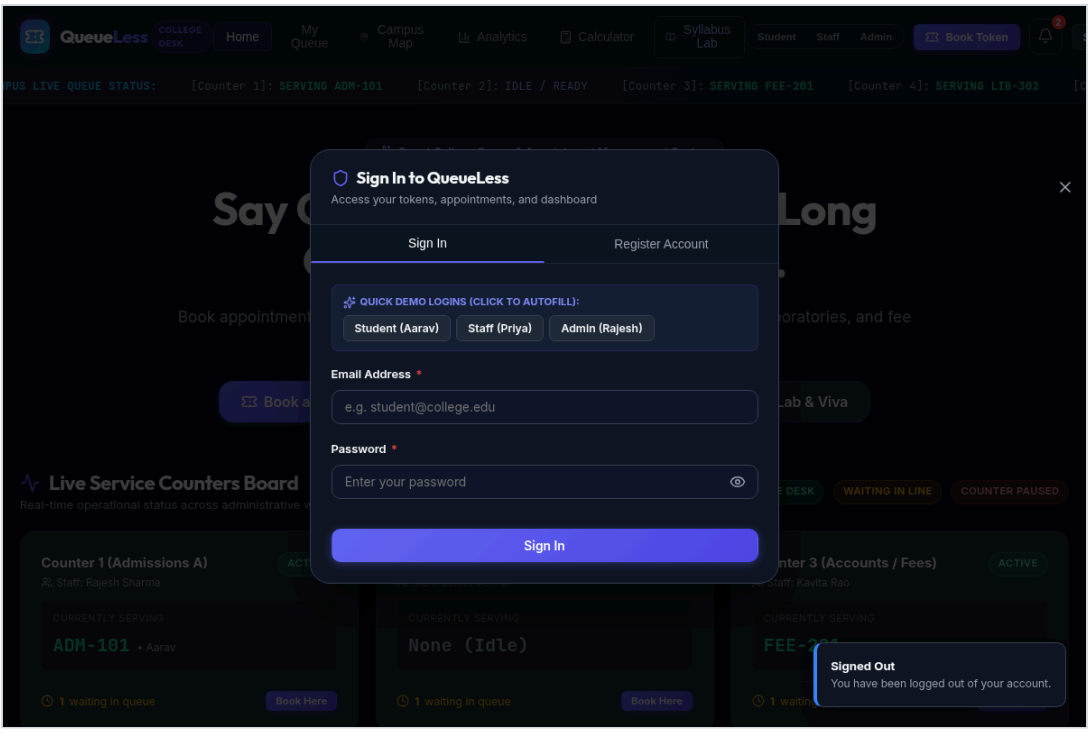
DASHBOARD PAGE:



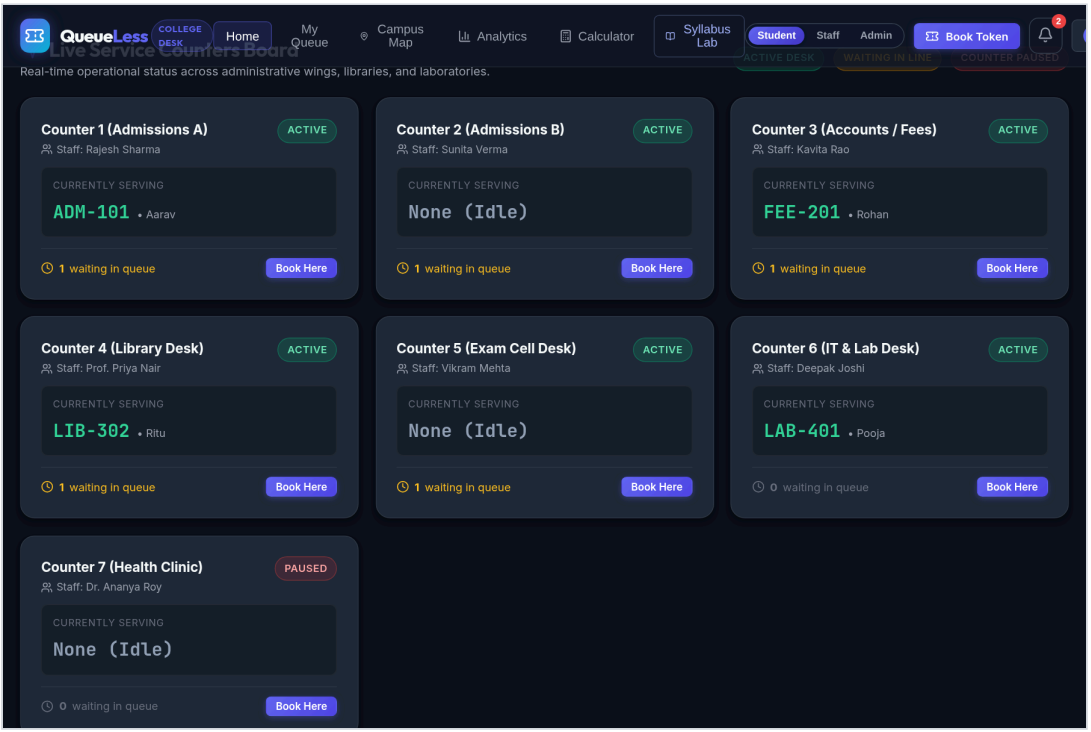
REGISTRATION PAGE:



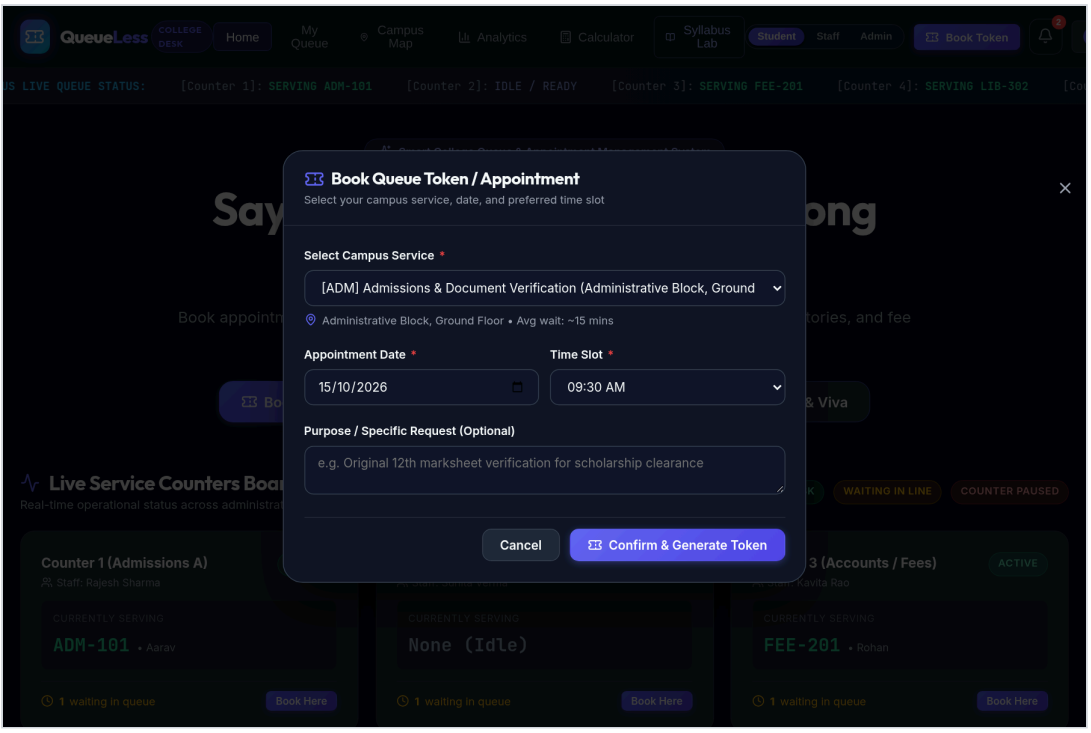
LOGIN PAGE:



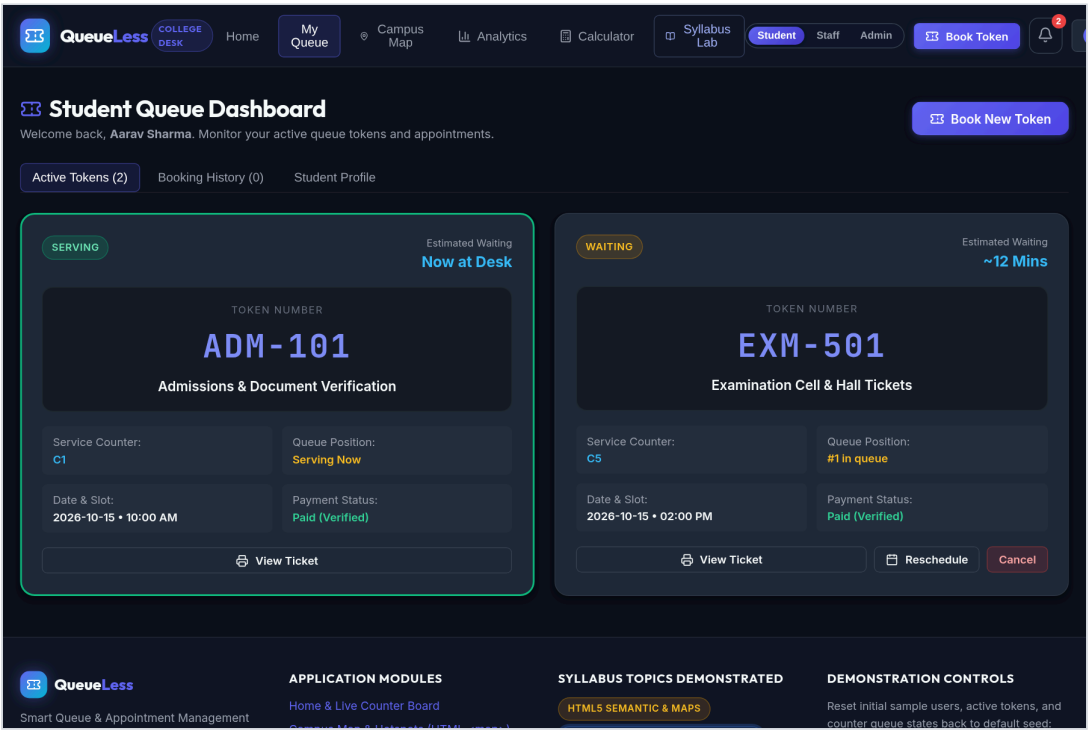
DESTINATION PAGE:



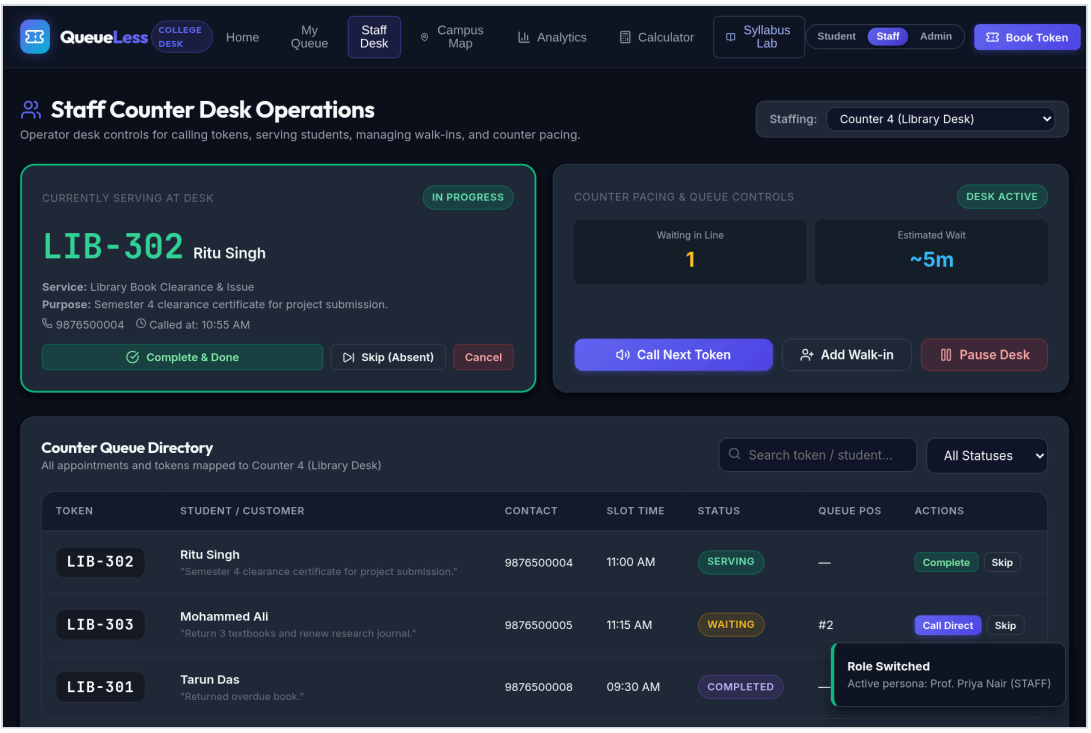
TOURIST PLACES PAGE:



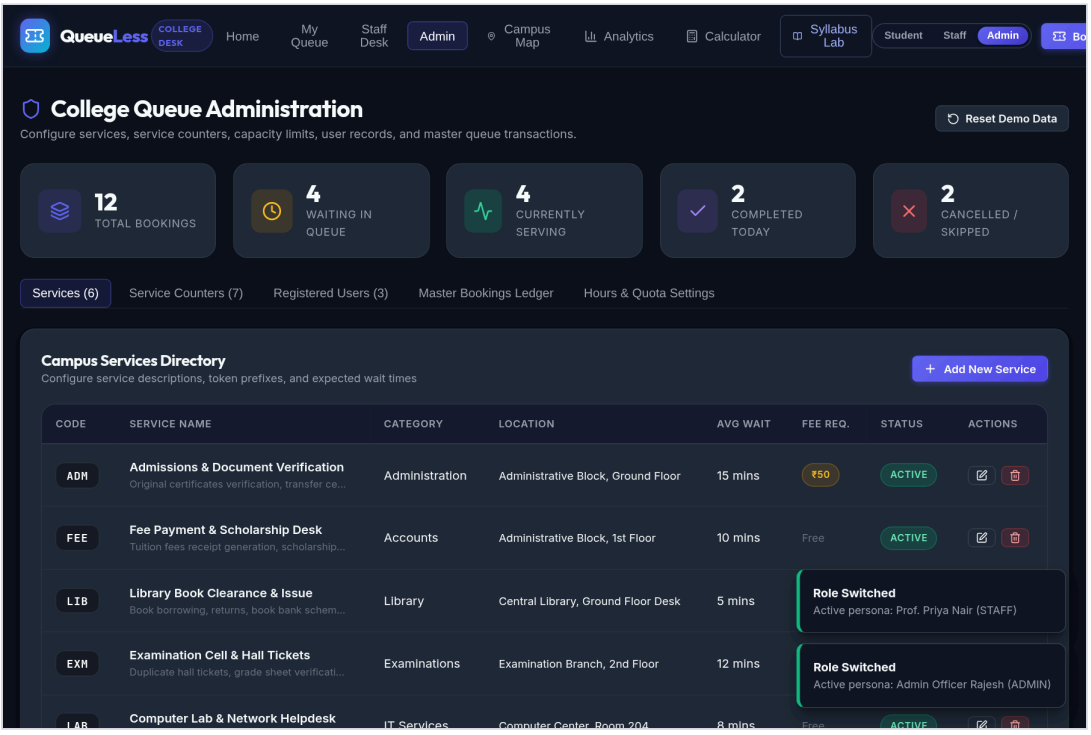
DURATION PAGE:



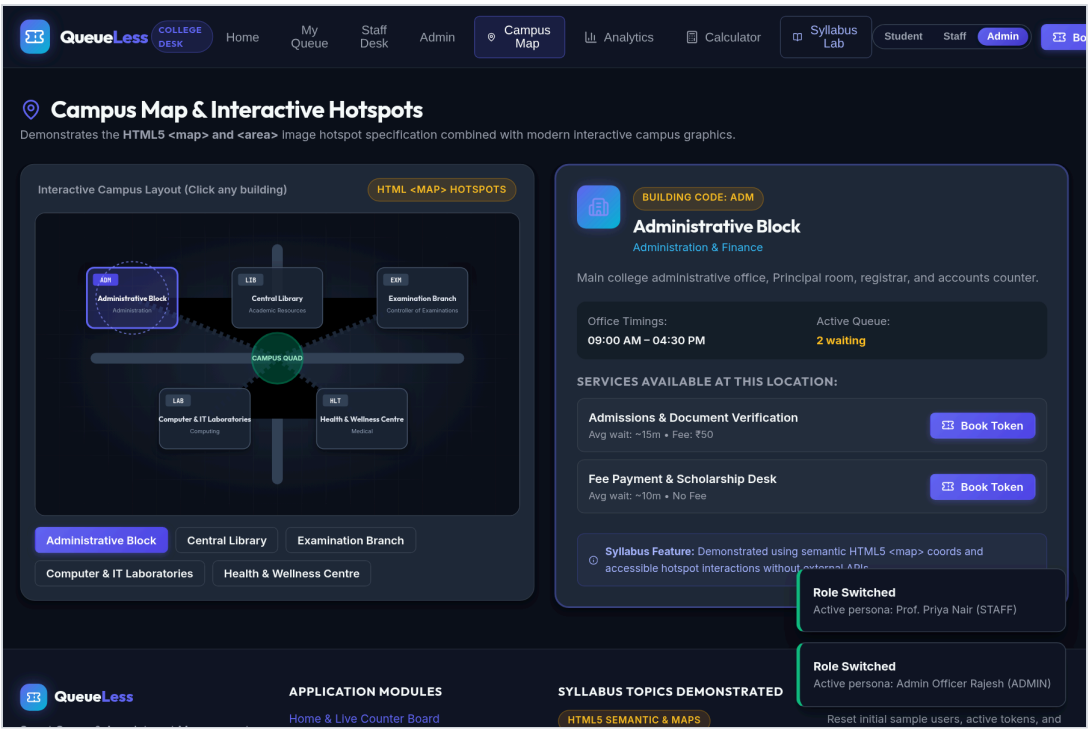
AVAILABLE GUIDES PAGE:



REQUESTED PAGE :



REVIEW PAGE :



CALCULATOR PAGE :

